

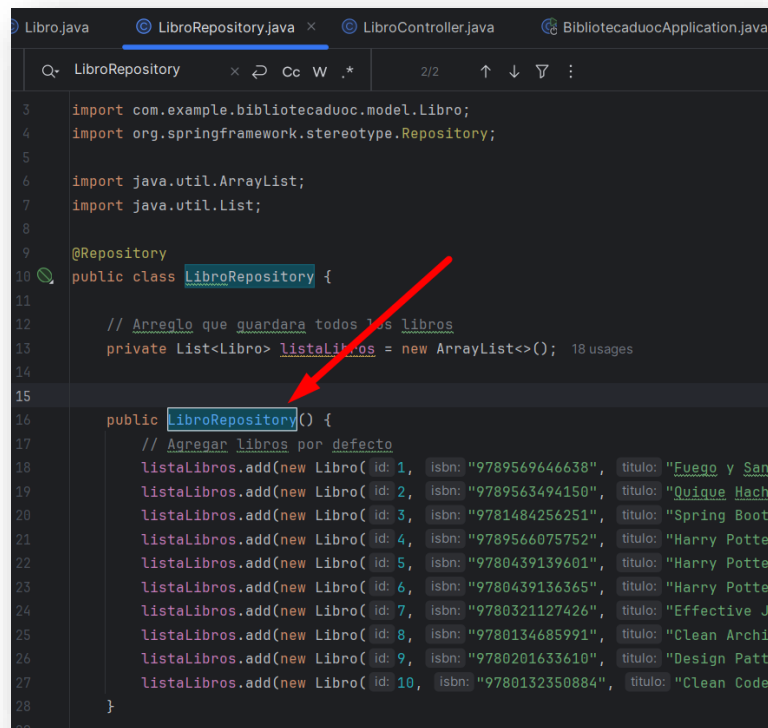
Guía práctica:

Bibliotecas Duoc – Reportes

1. CALCULAR EL TOTAL DE LIBROS QUE EXISTEN
2. ACTIVIDAD

Configuración

Agregar 10 nuevos libros en **LibroRepository**



```

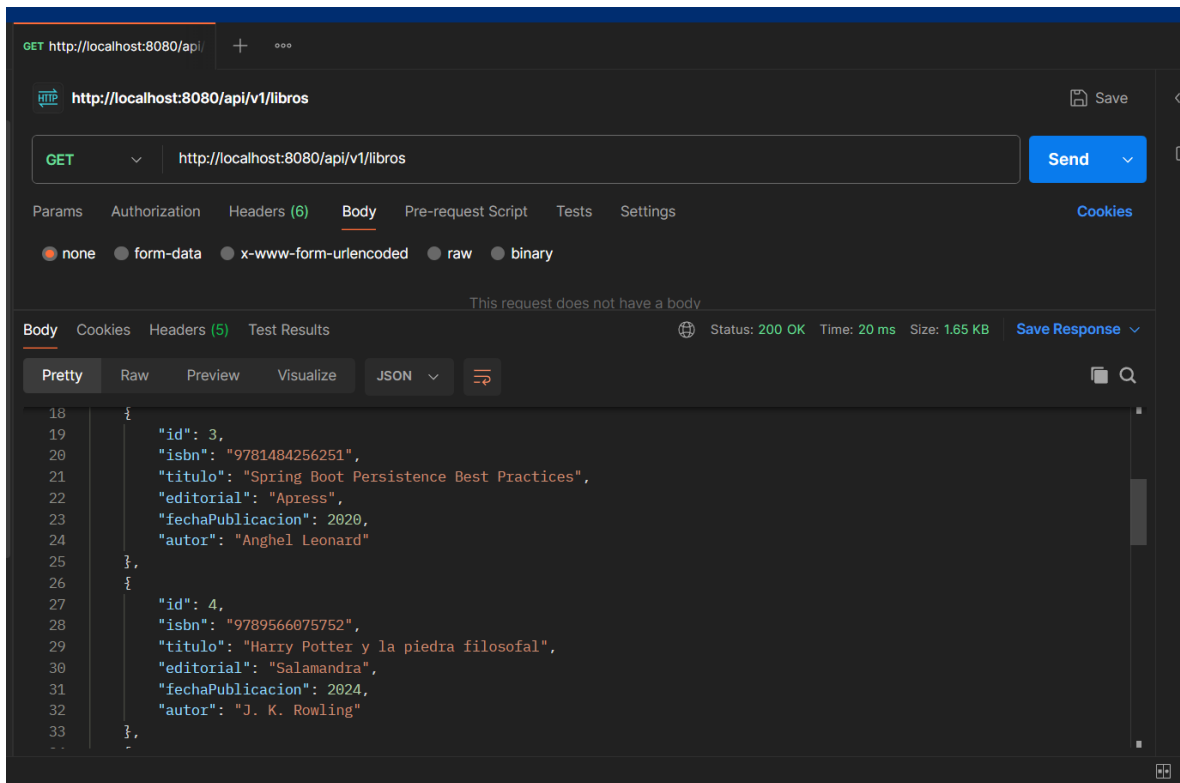
3  import com.example.bibliotecaduoc.model.Libro;
4  import org.springframework.stereotype.Repository;
5
6  import java.util.ArrayList;
7  import java.util.List;
8
9  @Repository
10 public class LibroRepository {
11
12     // Arreglo que guardara todos los libros
13     private List<Libro> listaLibros = new ArrayList<>(); 18 usages
14
15
16     public LibroRepository() {
17         // Agregar libros por defecto
18         listaLibros.add(new Libro(1, "9789569646638", "Fuego y Sangre", "Penguin Random House
19         listaLibros.add(new Libro(2, "9789563494150", "Quique Hache: El Mall Embrujado
20         listaLibros.add(new Libro(3, "9781484256251", "Spring Boot Persistence Best
21         listaLibros.add(new Libro(4, "9789566075752", "Harry Potter y la piedra
22         listaLibros.add(new Libro(5, "9780439139601", "Harry Potter y el prisionero de
23         listaLibros.add(new Libro(6, "9780439136365", "Harry Potter y el cáliz de
24         listaLibros.add(new Libro(7, "9780321127426", "Effective Java", "Addison-
25         listaLibros.add(new Libro(8, "9780134685991", "Clean Architecture", "Prentice
26         listaLibros.add(new Libro(9, "9780201633610", "Design Patterns", "Addison-
27         listaLibros.add(new Libro(10, "9780132350884", "Clean Code", "Robert C. Martin"));
28     }
  
```

```

listaLibros.add(new Libro(1, "9789569646638", "Fuego y Sangre", "Penguin Random House
Grupo Editorial", 2018, "George R. R. Martin"));
listaLibros.add(new Libro(2, "9789563494150", "Quique Hache: El Mall Embrujado
y Otras Historias", "Sm Ediciones", 2014, "Sergio Gomez"));
listaLibros.add(new Libro(3, "9781484256251", "Spring Boot Persistence Best
Practices", "Apress", 2020, "Anghel Leonard"));
listaLibros.add(new Libro(4, "9789566075752", "Harry Potter y la piedra
filosofal", "Salamandra", 2024, "J. K. Rowling"));
listaLibros.add(new Libro(5, "9780439139601", "Harry Potter y el prisionero de
Azkaban", "Scholastic", 1999, "J. K. Rowling"));
listaLibros.add(new Libro(6, "9780439136365", "Harry Potter y el cáliz de
fuego", "Scholastic", 2000, "J. K. Rowling"));
listaLibros.add(new Libro(7, "9780321127426", "Effective Java", "Addison-
Wesley", 2008, "Joshua Bloch"));
listaLibros.add(new Libro(8, "9780134685991", "Clean Architecture", "Prentice
  
```

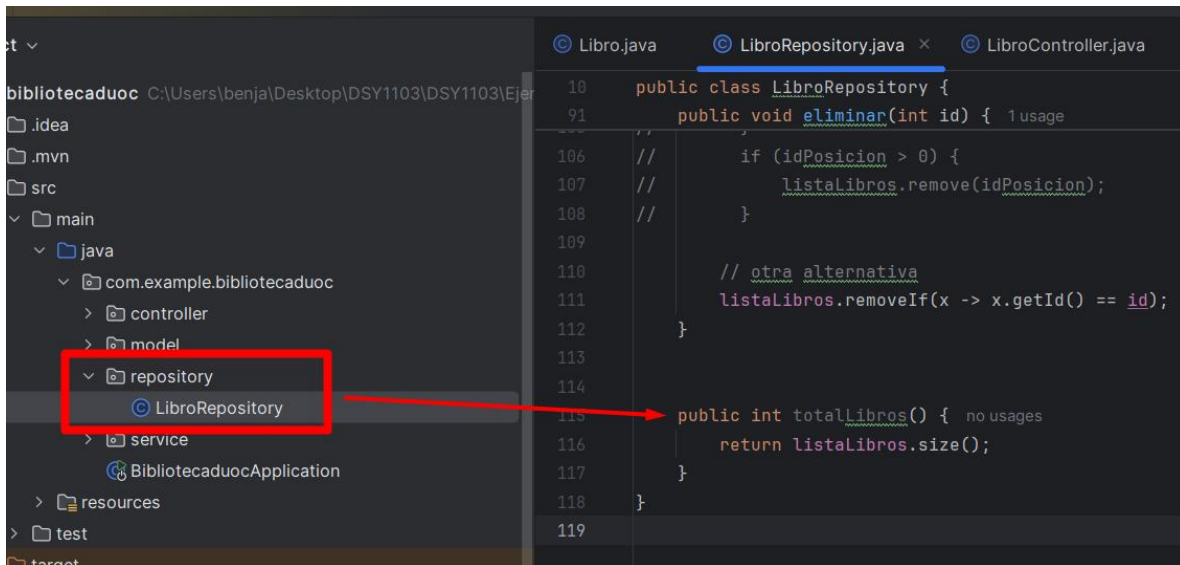
```
Hall", 2017, "Robert C. Martin"));
    listaLibros.add(new Libro(9, "9780201633610", "Design Patterns", "Addison-
Wesley", 1994, "Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides"));
    listaLibros.add(new Libro(10, "9780132350884", "Clean Code", "Prentice Hall",
2008, "Robert C. Martin"));
```

Al iniciar el repositorio, el método constructor añadirá 10 libros al arreglo.

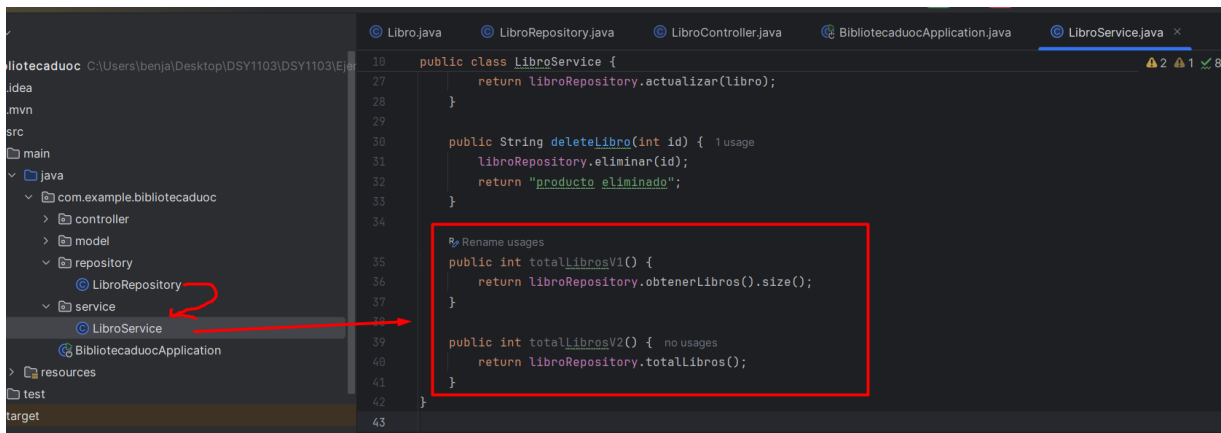


CALCULAR EL TOTAL DE LIBROS QUE EXISTEN

- Agregar en **LibroRepository** el método **totalLibros()** con retorno a un número entero.



- Luego el **LibroService** se encargará de entregar el total.



Pero tenemos el siguiente dilema con la arquitectura.

¿De quién es la responsabilidad de calcular el total?

REPOSITORY o SERVICE

En una arquitectura típica de **Spring**, la responsabilidad de calcular el total de libros (**o cualquier otra operación de negocio**) recae principalmente en el **servicio (Service)**. El servicio coordina las llamadas a los repositorios y encapsula la lógica de negocio.

Resumen de cada uno:

Capa de Repositorio (Repository)

El repositorio debe encargarse de las operaciones **CRUD básicas (Crear, Leer, Actualizar, Eliminar)** y cualquier consulta directa a la base de datos o lista en memoria. En este caso, podría tener un método para contar el número de libros.

Capa de Servicio (Service)

El servicio se encarga de la lógica de negocio y de coordinar las operaciones entre diferentes repositorios si es necesario. También es el lugar donde se agregarían transformaciones o lógica adicional antes de devolver los datos a la capa de presentación (controlador).

Solución

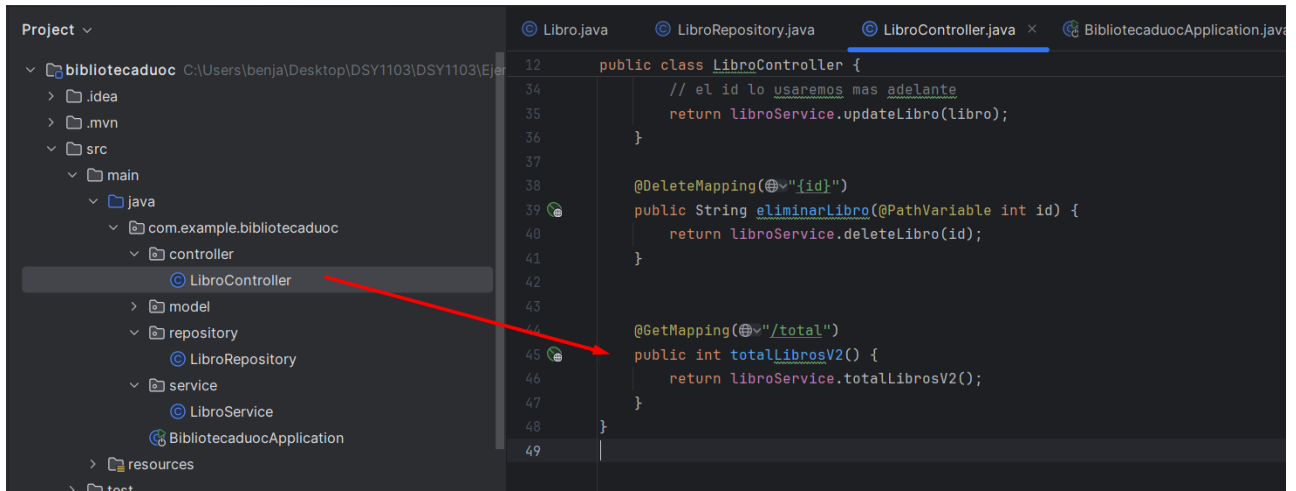
- **Capa de Repositorio:** Debería proporcionar un método para obtener el total de libros. Esto es simplemente una operación de lectura.
- **Capa de Servicio:** Debería utilizar el método del repositorio para obtener el total de libros y podría agregar cualquier lógica adicional si fuera necesario.

En este caso podemos dejar las dos formas

```
// LA ACCIÓN LA HACE EL SERVICE
public int totalLibros() { no usages
    return libroRepository.obtenerLibros().size();
}

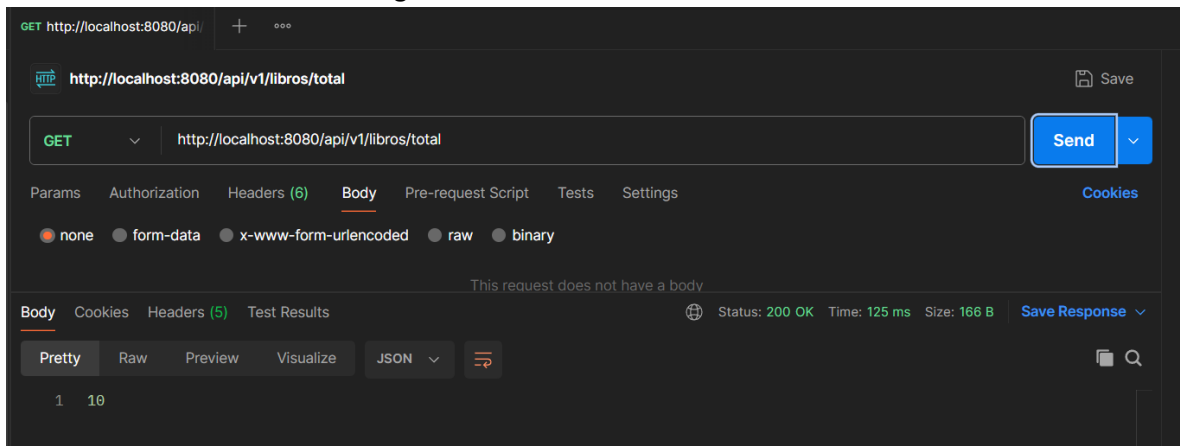
// LA ACCIÓN LA HACE EL MODELO
public int totalLibrosV2() { 1 usage
    return libroRepository.totalLibros();
}
```

- Agregar en **LibroController** el método **totalLibrosV2()** con retorno a un número entero.
 - El **GetMapping("/total")**



Probamos el nuevo **endpoint** en POSTMAN y vemos que funciona correctamente.

- Como retorno nos entrega el valor 10.



Actividades de reportes

Desarrolla los siguientes métodos personalizados en tu proyecto.

- Buscar un libro por isbn:
 - permite encontrar un libro específico utilizando su isbn.
- Buscar por cantidad de libros en un año específico:
 - permite encontrar el total de libros que se publicaron ese año.
- Buscar por autor:
 - permite encontrar libros escritos por un autor específico.
- Buscar el libro más antiguo.
 - Permite mediante la fecha de publicación comparar hasta encontrar el más antiguo.
- Buscar el libro más nuevo.
 - Permite mediante la fecha de publicación comparar hasta encontrar el más nuevo.
- Listar todos los libros ordenados por año de publicación:
 - permite obtener una lista de libros ordenados cronológicamente.

